

A project report
On
CareerStack
An AI-Powered Career Development & Skill Tracking Platform

Submitted to
IILM UNIVERSITY
In Partial Fulfilment for the award of the degree of

B.TECH
In
COMPUTER SCIENCE AND ENGINEERING

By
Shivam (CS-2341121)
Saurav Bhatt (CS-2341130)
Ashutosh Gupta (CS-2341638)
Under the guidance of
Mr.Shobit Agarwal
Assistant Professor in SCSE



IILM UNIVERSITY,
16-18, Knowledge Park II,
Greater Noida
Uttar Pradesh 201306

February 2026

DECLARATION

I declare that the project entitled "CareerStack: An AI-Powered Career Development & Skill Tracking Platform" has been prepared by me under the guidance of Mr.Shobit Agarwal. No part of this project has formed the basis for the award of any degree or fellowship previously.

This project is an original contribution to the field of career development technology, built using modern web technologies including Node.js, Express.js, MongoDB, and vanilla JavaScript. All third-party libraries and resources used have been duly acknowledged.

Student Name : Shivam , Saurav Bhatt, Ashutosh Gupta

Student Roll Number : CS-2341121 , CS-23411130, CS-2341638

CERTIFICATE

This is to certify that the project entitled "CareerStack: An AI-Powered Career Development & Skill Tracking Platform" is a bonafide work carried out by Shivam(CS-2341121), Saurav Bhatt(CS-23411130), Ashutosh Gupta(CS-2341638), students of B.Tech in Computer Science and Engineering at IILM University, Greater Noida, in partial fulfilment for the award of Bachelor of Technology degree.

This project has been completed under my supervision and guidance. It is an original work and has not been submitted elsewhere for the award of any degree or diploma.

Faculty Guide: Mr.Shobit Agarwal

Designation: Assistant Professor in SCSE

Date: 5th May 2026

Department of Computer Science & Engineering
IILM University, Greater Noida

ABSTRACT

CareerStack is a full-stack web application developed to bridge the gap between students' existing skill sets and the evolving demands of the technology job market. Aligned with the United Nations Sustainable Development Goal 8 (SDG-8) — Decent Work and Economic Growth — the platform provides an integrated ecosystem for career skill development, income tracking, and intelligent job matching.

The system is built on a RESTful architecture using Node.js and Express.js for the backend, MongoDB as the NoSQL database, and a multi-page vanilla JavaScript frontend styled with a responsive custom CSS design system. Authentication is handled via JSON Web Tokens (JWT), with role-based access control distinguishing between regular users and administrators.

Core features of CareerStack include: (1) a dynamic Job Feed that displays real-time job listings stored in MongoDB; (2) a Skill Match Engine that compares a user's declared skills against job requirements and produces a quantified match percentage; (3) an intelligent Course Recommendation System that suggests relevant learning resources for missing skills; (4) an Income Tracker that records and visualises monthly earnings over time using Chart.js; and (5) an Admin Dashboard enabling authorised personnel to manage jobs, courses, and platform statistics.

The platform was seeded with 6 representative job roles and 16 free online courses from providers including Coursera, YouTube, and Amazon, all mapped to specific technical skills. Security mechanisms include bcrypt password hashing, JWT-based stateless authentication, and protected admin routes. The system demonstrates significant potential for deployment as a career readiness tool in academic institutions.

Keywords: Career Development, Skill Matching, Job Recommender System, Income Tracker, RESTful API, Node.js, Express.js, MongoDB, JWT Authentication, SDG-8.

ACKNOWLEDGMENT

We take this opportunity to express our heartfelt gratitude to all those who have contributed to the successful completion of this project.

We are profoundly grateful to our Faculty Guide, Mr. Shobit Agarwal, assistant professor in SCSE, Department of Computer Science and Engineering, IILM University, for their invaluable guidance, continuous encouragement, and insightful feedback throughout the development of CareerStack. Their technical expertise and constructive critique shaped the direction of this project significantly.

We extend our sincere thanks to the Head of Department, and all faculty members of the Computer Science and Engineering department whose teachings provided us with the foundational knowledge necessary for this project.

We are grateful to the IILM University administration for providing the necessary infrastructure, computing resources, and a conducive environment for academic research and project development.

Special thanks are due to the open-source community — particularly the developers of Node.js, Express.js, MongoDB, Chart.js, and the docx.js ecosystem — whose freely available tools made this project possible.

Finally, we express our deep gratitude to our families and friends for their unwavering moral support, patience, and encouragement throughout this academic journey.

Shivam (CS-2341121)
Saurav Bhatt (CS-23411130)
Ashutosh Gupta (CS-2341638)
B.Tech CSE — IILM University

TABLE OF CONTENTS

DECLARATION	1
CERTIFICATE	3
ABSTRACT	4
ACKNOWLEDGMENT	5
TABLE OF CONTENTS	6
LIST OF FIGURES	8
LIST OF TABLES	9
LIST OF ABBREVIATIONS	10
CHAPTER 1: INTRODUCTION	11
1.1 Background and Motivation	11
1.2 Problem Statement	11
1.3 Objectives	11
1.4 Scope of the Project	12
1.5 SDG Alignment	12
1.6 Report Organisation	12
CHAPTER 2: LITERATURE STUDY	14
2.1 Overview	14
2.2 Career Development Platforms — Existing Systems	14
2.3 Skill Gap Analysis — Related Research	14
2.4 Income Tracking and Financial Literacy Tools	15
2.5 Technology Stack Justification	15
2.6 Research Gap and CareerStack Contribution	15
CHAPTER 3: DATASET PREPARATION AND PRE-PROCESSING	17
3.1 Data Architecture Overview	17
3.2 MongoDB Collections and Schemas	17
3.2.1 User Schema	17
3.2.2 Job Schema	17
3.2.3 Course Schema	17
3.2.4 Income Schema	17
3.3 Seed Data Preparation	17
3.3.1 seedAdmin.js	18
3.3.2 seedData.js	18
3.4 Data Validation and Pre-Processing	19
3.5 Database Connection	19

CHAPTER 4: METHODOLOGY & NOVELTY	20
4.1 System Architecture.....	20
4.2 Authentication and Authorisation Methodology.....	20
4.2.1 Registration Flow.....	20
4.2.2 Login Flow.....	20
4.2.3 Protected Routes	20
4.3 Skill Match Engine — Core Algorithm.....	20
4.4 Income Tracker Implementation.....	21
4.5 Admin Dashboard.....	21
4.6 API Design.....	22
4.7 Novelty and Contributions.....	23
4.8 Security Implementation.....	23
CHAPTER 5: EXPERIMENTAL RESULTS AND DISCUSSION	24
5.1 Test Environment.....	24
5.2 Authentication System Results	24
5.3 Skill Match Engine Results.....	24
5.4 Income Tracker Results	25
5.5 API Performance.....	26
5.6 UI/UX Testing	26
5.7 Discussion	26
CHAPTER 6: CONCLUSIONS AND FUTURE DIRECTIONS	28
6.1 Conclusions.....	28
6.2 Limitations	28
6.3 Future Directions	29
6.4 Final Remarks	29
REFERENCES	30

LIST OF FIGURES

- Figure 1.1** CareerStack System Architecture Overview
- Figure 1.2** SDG Goal 8 Alignment Diagram
- Figure 3.1** Entity Relationship (ER) Diagram of MongoDB Collections
- Figure 3.2** User Schema Data Model
- Figure 3.3** Income Schema Data Model
- Figure 4.1** System Architecture — MPA Frontend and REST Backend
- Figure 4.2** JWT Authentication Flow
- Figure 4.3** Skill Match Algorithm Flowchart
- Figure 4.4** Income Tracker — Chart.js Line Graph
- Figure 5.1** Login Page Screenshot
- Figure 5.2** User Dashboard Screenshot
- Figure 5.3** Job Feed with Skill Filter Screenshot
- Figure 5.4** Skill Match Result — 75% Match Example
- Figure 5.5** Income Tracker with Monthly Chart
- Figure 5.6** Admin Dashboard — Job & Course Management

LIST OF TABLES

Table 2.1 Comparison of Existing Career Development Platforms

Table 3.1 MongoDB Collections and Document Fields

Table 3.2 Seed Data — Job Roles and Required Skills

Table 3.3 Seed Data — Courses and Skill Tags

Table 4.1 API Endpoints — Authentication Routes

Table 4.2 API Endpoints — Jobs & Courses Routes

Table 4.3 API Endpoints — Income Routes

Table 4.4 API Endpoints — Admin Routes

Table 5.1 Skill Match Test Cases and Results

Table 5.2 Platform Performance Metrics

LIST OF ABBREVIATIONS

Abbreviation	Full Form
API	Application Programming Interface
REST	Representational State Transfer
JWT	JSON Web Token
MPA	Multi-Page Application
MERN	MongoDB, Express.js, React, Node.js
CRUD	Create, Read, Update, Delete
SDG	Sustainable Development Goal
HTML	HyperText Markup Language
CSS	Cascading Style Sheets
JS	JavaScript
DB	Database
ORM	Object Relational Mapping
ODM	Object Document Mapping
HTTP	HyperText Transfer Protocol
CORS	Cross-Origin Resource Sharing
SHA	Secure Hash Algorithm
bcrypt	Password-Hashing Function
npm	Node Package Manager
env	Environment Variable
UI	User Interface
UX	User Experience

CHAPTER 1: INTRODUCTION

1.1 Background and Motivation

The global employment landscape is undergoing rapid transformation driven by technological advancements, digital disruption, and the growing mismatch between academic skill development and industry demands. In India alone, millions of engineering graduates enter the workforce each year, yet a significant proportion remain underemployed due to inadequate alignment between their technical competencies and the actual skill requirements of employers.

This skills gap is not merely an academic issue — it has profound socioeconomic consequences, directly undermining the achievement of the United Nations Sustainable Development Goal 8 (SDG-8), which advocates for sustained, inclusive, and sustainable economic growth, full and productive employment, and decent work for all.

CareerStack was conceived as a direct response to this challenge. It is a full-stack web application designed to help students and early-career professionals identify skill gaps, discover targeted learning resources, track career progression, and monitor financial growth — all within a single, cohesive digital platform.

1.2 Problem Statement

Despite the widespread availability of online job postings and learning resources, there is no unified, personalised platform that:

- Quantifies how well a student's current skills match a specific job role's requirements
- Recommends specific, free, curated courses to bridge identified skill gaps
- Tracks income growth over time to provide financial awareness and career progress visibility
- Provides administrators with tools to manage job listings and course catalogues dynamically

Existing platforms such as LinkedIn, Coursera, and Naukri address some of these needs in isolation, but none integrates them into a single, lightweight, role-based system accessible to students from any device without institutional subscriptions or paywalls.

1.3 Objectives

The primary objectives of CareerStack are:

1. To design and implement a responsive, multi-page web application for career skill development
2. To build a RESTful API backend with Node.js and Express.js, backed by MongoDB

3. To implement a JWT-based authentication system with role-based access control (user/admin)
4. To develop a Skill Match Engine that computes and displays the percentage match between user skills and job requirements
5. To build an intelligent Course Recommendation System that ranks courses by coverage of missing skills
6. To implement an Income Tracker with visual Chart.js-based monthly earnings graphs
7. To create an Admin Dashboard for managing job listings, course catalogues, and platform statistics
8. To align the platform with SDG Goal 8 — Decent Work and Economic Growth

1.4 Scope of the Project

CareerStack encompasses both a frontend (client-side) and backend (server-side) system:

Frontend Scope: Seven HTML pages (index, login, register, user-dashboard, jobfeed, income tracker, admin-dashboard), one shared CSS design system (style.css), and three shared JavaScript modules (auth.js, theme.js, skillmatch.js). The UI supports dark/light theming, responsive mobile-first layouts, and dynamic data rendering via vanilla JavaScript fetch calls to the REST API.

Backend Scope: A Node.js/Express.js REST API with five route modules (auth, jobs, courses, income, admin), three Mongoose models (User, Job, Course, Income), two middleware modules (JWT protect, adminOnly), a database seeder (seedData.js for jobs and courses), and an admin seeder (seedAdmin.js).

Out of Scope: Real-time chat, OAuth social login implementation, resume parser, video course hosting, payment gateway integration, and mobile native apps — these are identified as future enhancements.

1.5 SDG Alignment

CareerStack directly supports SDG Goal 8 by:

- Improving the quality and relevance of skill development resources available to youth
- Providing a transparent, merit-based skill assessment tool to reduce information asymmetry in the job market
- Enabling income tracking that fosters financial literacy and economic self-awareness
- Lowering the barrier to quality career guidance by making the platform free and accessible

1.6 Report Organisation

The remainder of this report is organised as follows: Chapter 2 presents a review of existing career platforms and related literature. Chapter 3 describes the dataset preparation and system data models. Chapter 4 details the methodology, system architecture, and implementation. Chapter 5 presents experimental results and evaluation. Chapter 6 concludes the report and outlines future directions.

CHAPTER 2: LITERATURE STUDY

2.1 Overview

This chapter reviews existing work in three related domains: career skill development platforms, skill gap analysis systems, and income tracking tools. It also examines the technology stack choices made for CareerStack in the context of existing research.

2.2 Career Development Platforms — Existing Systems

Several prominent platforms address aspects of career development:

Platform	Key Features	Limitations
LinkedIn Learning	Course catalogue, Job board, Skill endorsements	Paywalled, No personalised skill gap analysis
Coursera	University courses, Certifications, Career paths	No job matching, No income tracking
Naukri.com	Job listings, Resume builder, Skill tests	No course recommendations, India-centric
HackerRank	Coding assessments, Skill certifications	Only technical coding skills, No income tracker
CareerStack (Ours)	Skill match %, Course recommendations, Income tracker, Admin CMS	Early-stage, no OAuth, no resume parser

The comparison table (Table 2.1) illustrates that no single existing platform combines the three core pillars of CareerStack: quantified skill matching, personalised course recommendations, and income progression tracking within a single, accessible, and lightweight application.

2.3 Skill Gap Analysis — Related Research

Skill gap analysis has been studied extensively in educational technology and human resource management literature. Research by Acemoglu and Autor (2011) established the Task-Based Model of the labour market, demonstrating how technological change creates persistent skill mismatches. Subsequent work by Deming (2017) highlighted the growing premium on social and cognitive skills in the modern labour market.

In the domain of educational technology, recommender systems for learning resources have been explored through collaborative filtering, content-based filtering, and hybrid approaches. The skill match algorithm implemented in CareerStack employs a simplified content-based filtering approach — comparing user-

declared skills against job-required skills using set intersection, producing a match percentage (Jaccard-inspired coefficient), and recommending courses ranked by coverage of identified skill gaps.

This lightweight approach, while less sophisticated than machine-learning-based recommenders, is well-suited to the resource constraints of a student project while delivering measurable, interpretable results.

2.4 Income Tracking and Financial Literacy Tools

Income tracking applications have been studied in the context of financial literacy education. Research consistently demonstrates that visualising income growth improves financial decision-making and long-term career planning behaviour among young adults. Tools such as Mint, YNAB, and Money Manager implement personal finance tracking, but none integrate income tracking with career skill development.

CareerStack fills this gap by embedding an income tracker directly within the career development workflow, allowing users to correlate income growth with skill acquisition milestones.

2.5 Technology Stack Justification

The technology stack selected for CareerStack is grounded in practical research and industry adoption trends:

Technology	Justification
Node.js	Non-blocking I/O, widely used for API servers, extensive npm ecosystem
Express.js	Minimal, unopinionated, industry-standard web framework for Node.js
MongoDB	Schema-flexible NoSQL DB ideal for evolving data models in skill/job domains
Mongoose ODM	Schema validation, middleware hooks, and type casting for MongoDB
JWT	Stateless, scalable token-based auth — no server-side session storage required
bcryptjs	Industry-standard adaptive hashing algorithm for password security
Chart.js	Lightweight, responsive charting library for income visualisation
Vanilla JS	Zero dependency, universal browser support, suitable for MPA architecture
CSS Variables	Enables dark/light theme switching without JavaScript framework overhead

2.6 Research Gap and CareerStack Contribution

Existing literature confirms that while individual components of career development tools have been studied extensively, there is a significant absence of integrated, lightweight, open-access systems that bring together skill gap analysis, personalised learning recommendation, income tracking, and role-based

administrative management into a single platform deployable without institutional infrastructure. CareerStack addresses this gap.

CHAPTER 3: DATASET PREPARATION AND PRE-PROCESSING

3.1 Data Architecture Overview

CareerStack uses MongoDB as its primary data store, accessed via the Mongoose ODM library. The database consists of four collections: Users, Jobs, Courses, and Incomes. Data is structured using Mongoose schemas that enforce types, constraints, default values, and relationships.

3.2 MongoDB Collections and Schemas

3.2.1 User Schema

The User model (`models/User.js`) stores authentication and role information. Fields include: `name` (String, required), `email` (String, required, unique, lowercase), `password` (String, required, min 6 chars, select: false), and `role` (enum: `user|admin`, default: `user`). Timestamps (`createdAt`, `updatedAt`) are automatically managed.

Security preprocessing is applied via a Mongoose pre-save middleware hook that automatically hashes passwords using `bcryptjs` with a salt factor of 10 before persisting to the database. A `matchPassword()` instance method enables secure comparison during login without requiring password field exposure.

3.2.2 Job Schema

The Job model (`utils/Job.js`) stores job listing data. Fields include: `title` (String, required), `company` (String, required), `location` (String, default: `Remote`), `salary` (String), `type` (String, default: `Full-time`), `description` (String), `skills` ([String], array of skill tags), `applyUrl` (String), and `createdBy` (ObjectId reference to User).

3.2.3 Course Schema

The Course model (`models/Course.js`) stores learning resource data. Fields include: `title`, `provider`, `category`, `price` (Number, default: 0), `image` (URL), `link` (course URL), `description`, `skills` ([String], array of skill tags for recommendation matching), and `level` (enum: `beginner|intermediate|advanced`).

3.2.4 Income Schema

The Income model (`models/Income.js`) stores user income records. Fields include: `user` (ObjectId reference to User, required), `source` (String, required — e.g., `Internship`, `Freelance`, `Salary`), `amount` (Number, required, min: 0), `category` (enum: `freelance|internship|part-time|stipend|other`), `note` (String, max 300 chars), and `date` (Date, default: `now`).

3.3 Seed Data Preparation

To ensure the platform is functional upon deployment, two seed scripts were developed:

3.3.1 seedAdmin.js

Checks if an admin user exists (by email from environment variable ADMIN_EMAIL) and creates one if not. Default credentials are: email=admin@careerstack.com, password=Admin@123. This script is executed automatically on every server startup, making deployment idempotent.

3.3.2 seedData.js

Populates the Jobs and Courses collections with curated data. It is executed manually via node scripts/seedData.js after database connection. The seed data was prepared based on industry research into the most in-demand tech roles in 2025-26.

Seed Job Roles (6):

Job Title	Company	Required Skills
AI / ML Engineer	OpenAI Careers	Python, TensorFlow, PyTorch, ML Algorithms, NumPy, Pandas
Data Scientist	DataWorks	Python, SQL, Statistics, Pandas, NumPy, Power BI
Cybersecurity Analyst	SecureNet	Network Security, SIEM, Python, Linux, Risk Assessment
Cloud Engineer	CloudOps Inc	AWS, Docker, Kubernetes, Terraform, Linux
Full Stack Developer	CareerStack Labs	HTML, CSS, JavaScript, React, Node.js, MongoDB, REST API, Git
DevOps Engineer	InfraFlow	CI/CD, Jenkins, Docker, Kubernetes, AWS, Linux

Seed Courses (16):

Course Title	Provider	Skill Tags	Price
Python for Everybody	Coursera	Python	Free
TensorFlow in Practice	Coursera	TensorFlow, Python	Free
PyTorch Deep Learning	YouTube	PyTorch, Python	Free
Machine Learning by Andrew Ng	Coursera	ML Algorithms, Python	Free
SQL for Data Science	Coursera	SQL	Free
Pandas Tutorial	YouTube	Pandas, Python	Free
Statistics with Python	Coursera	Statistics, Python	Free
AWS Cloud Practitioner	Amazon	AWS	Free
Docker Masterclass	YouTube	Docker	Free
Kubernetes for Beginners	YouTube	Kubernetes	Free

Terraform Full Course	YouTube/freeCodeCamp	Terraform	Free
React JS Full Course	YouTube	React, JavaScript	Free
Node.js Full Course	YouTube	Node.js, JavaScript	Free
MongoDB Full Course	YouTube	MongoDB	Free
CI/CD with GitHub Actions	YouTube	CI/CD	Free
Python Full Course	YouTube/freeCodeCamp	Python	Free

3.4 Data Validation and Pre-Processing

The following validation and preprocessing steps are applied:

- Email normalisation: all emails are converted to lowercase and trimmed before storage or lookup, preventing duplicate accounts due to case variations
- Skill array normalisation: skills input as comma-separated strings are split, trimmed, and filtered for empty values before storage — applied in both admin job/course creation routes
- Skill matching normalisation: the skillmatch.js module normalises all skills to lowercase using normalizeSkill() before set comparison, ensuring case-insensitive matching (e.g., 'Python' matches 'python')
- Amount validation: income amounts are coerced to Number type and validated with min: 0 at schema level
- HTML escaping: all user-supplied text rendered in the frontend is passed through escapeHtml() to prevent XSS injection
- Password strength: client-side password strength indicator provides real-time feedback; server-side minimum length (6 chars) is enforced independently

3.5 Database Connection

The database connection is managed by config/db.js, which uses Mongoose's connect() method with the MONGO_URI environment variable. Connection failures terminate the process (process.exit(1)) to prevent the server from running in a degraded state. The connection is established at server startup before route handling begins.

CHAPTER 4: METHODOLOGY & NOVELTY

4.1 System Architecture

CareerStack follows a decoupled client-server architecture with a Multi-Page Application (MPA) frontend communicating with a RESTful Node.js/Express.js backend via HTTP API calls. The system architecture is divided into three tiers:

- Presentation Tier: Seven HTML pages with vanilla JavaScript, CSS design system (style.css), and three shared JS modules (auth.js, theme.js, skillmatch.js)
- Application Tier: Node.js + Express.js REST API server with five route modules, two middleware modules, and seed utilities
- Data Tier: MongoDB Atlas/local MongoDB instance accessed via Mongoose ODM

Communication between tiers uses JSON over HTTP. The frontend's auth.js module provides a centralised Auth.request() method that automatically attaches Bearer JWT tokens to all authenticated requests, eliminating the need for repetitive token-handling code across pages.

4.2 Authentication and Authorisation Methodology

4.2.1 Registration Flow

1. User submits name, email, password via register.html. 2. Client-side validation checks password match and minimum length. 3. POST /api/auth/register — server validates, checks email uniqueness, creates user. 4. Mongoose pre-save hook hashes password (bcrypt, salt=10). 5. JWT token generated (jwt.sign, 7-day expiry). 6. Token and user object returned; client stores in localStorage (cs_token, cs_user). 7. Redirect to user-dashboard.html.

4.2.2 Login Flow

1. User submits email and password via login.html. 2. POST /api/auth/login — server finds user by email (select +password), calls matchPassword(). 3. On success, JWT token issued and cached. 4. Role-based redirect: admin → admin-dashboard.html, user → user-dashboard.html.

4.2.3 Protected Routes

The protect middleware (middleware/auth.js) extracts the Bearer token from the Authorization header, verifies it using JWT_SECRET, fetches the user from MongoDB (excluding password), and attaches req.user. The adminOnly middleware additionally checks req.user.role === 'admin', returning HTTP 403 for unauthorised access attempts.

4.3 Skill Match Engine — Core Algorithm

The Skill Match Engine (skillmatch.js) implements the following algorithm:

9. Load all jobs from GET /api/jobs and populate the job dropdown
10. Load all courses from GET /api/courses for recommendation pool
11. User inputs their skills as a comma-separated string and selects a target job role
12. Parse user skills into an array; normalise all to lowercase via normalizeSkill()
13. Build a Set from user skills for O(1) lookup performance
14. Retrieve requiredSkills[] from the selected job document
15. Compute matchedSkills = requiredSkills filtered by presence in userSkillSet
16. Compute missingSkills = requiredSkills NOT in userSkillSet
17. Compute matchPercentage = (matchedSkills.length / requiredSkills.length) * 100
18. Render match level: Strong Match ($\geq 80\%$), Moderate Match ($\geq 60\%$), Needs Improvement ($\geq 40\%$), Low Match ($< 40\%$)
19. Course Recommendation: for each course, count how many of its skill tags appear in missingSkills; sort descending by coverage; show top 6

The course ranking algorithm is a lightweight content-based recommender using skill coverage count as the ranking signal. A course covering 3 missing skills ranks above one covering 1, maximising the learning efficiency per resource for the user.

4.4 Income Tracker Implementation

The income tracker (income.html + income.js) stores entries in localStorage (cs_incomeEntries key) for client-side persistence in the current implementation. Each entry records: month (YYYY-MM format), amount (Number), and source (String).

The system computes four summary statistics: (1) Average Monthly Income — mean of all amounts; (2) Highest Income — maximum amount; (3) Entry Count — total records; (4) Growth Percentage — $((\text{lastEntry} - \text{firstEntry}) / \text{firstEntry}) \times 100$, where entries are sorted chronologically by month.

Visualisation is implemented using Chart.js (line chart, tension: 0.35, responsive, beginAtZero). The chart is destroyed and re-rendered on every data update to prevent canvas reuse conflicts. The backend Income model (income.routes.js) provides a full REST API for server-side persistence — a migration path available for future deployment.

4.5 Admin Dashboard

The admin dashboard (admin-dashboard.html) is protected by requireAdmin() on the frontend (redirect to user-dashboard if role !== admin) and by protect + adminOnly middleware on all /api/admin/* routes on the backend.

Admin capabilities include:

- View platform statistics: total users, jobs, and courses (GET /api/admin/stats)
- Create new job listings with full field set including skill tags and apply URL (POST /api/admin/jobs)
- Delete existing job listings (DELETE /api/admin/jobs/:id)
- Create new courses with skill tags and course links (POST /api/admin/courses)
- Delete existing courses (DELETE /api/admin/courses/:id)

Skill tags for both jobs and courses accept comma-separated input and are pre-processed server-side into clean string arrays before storage.

4.6 API Design

The REST API follows standard conventions:

Method	Endpoint	Auth	Description
POST	/api/auth/register	None	Register new user
POST	/api/auth/login	None	Login and receive JWT
GET	/api/auth/me	JWT	Fetch current user profile
GET	/api/jobs	None	List all jobs (search/skill filter)
GET	/api/courses	None	List all courses (skill filter)
GET	/api/income	JWT	Get user income entries + total
POST	/api/income	JWT	Add income entry
DELETE	/api/income/:id	JWT	Delete income entry
GET	/api/admin/stats	JWT+Admin	Platform statistics
GET	/api/admin/jobs	JWT+Admin	List all jobs (admin)
POST	/api/admin/jobs	JWT+Admin	Create new job
DELETE	/api/admin/jobs/:id	JWT+Admin	Delete job
GET	/api/admin/courses	JWT+Admin	List all courses (admin)
POST	/api/admin/courses	JWT+Admin	Create new course

DELETE	/api/admin/courses/:id	JWT+Admin	Delete course
--------	------------------------	-----------	---------------

4.7 Novelty and Contributions

The novel aspects of CareerStack relative to existing systems include:

- **Integrated Skill-to-Course Coverage Ranking:** Unlike generic course catalogues, CareerStack ranks courses specifically by how many of a user's missing skills they cover — a personalised, dynamic ranking not seen in existing free platforms
- **Unified Career Intelligence Dashboard:** Combining skill gap analysis, income tracking, and job browsing in a single role-based platform is a novel integration for a lightweight, open-access web application
- **Idempotent Admin Seeding:** The seedAdmin.js utility ensures deployment idempotency — the admin account is only created if absent, preventing duplication across restarts
- **Dark/Light Theme System:** The CSS variable-based theme system (theme.js + style.css) provides universal theming across all seven pages with a single 5KB JavaScript module
- **Zero-Framework Frontend:** Delivering a feature-rich, multi-page application with no JavaScript framework (no React, Vue, or Angular) demonstrates architectural efficiency and broad accessibility

4.8 Security Implementation

- **Password Hashing:** bcryptjs with salt rounds = 10 — computationally expensive enough to resist brute-force attacks
- **JWT Expiry:** Tokens expire in 7 days; clearSession() removes tokens on logout
- **Protected Routes:** All sensitive endpoints require valid JWT; admin routes additionally require role=admin
- **Input Sanitisation:** escapeHtml() applied to all user-supplied text rendered in the DOM
- **CORS:** Configurable via FRONTEND_ORIGIN environment variable; defaults to * for development
- **Error Handling:** Centralised errorHandler middleware returns structured JSON errors without leaking stack traces to clients

CHAPTER 5: EXPERIMENTAL RESULTS AND DISCUSSION

5.1 Test Environment

All testing was performed on a local development environment with the following configuration:

Backend Runtime	Node.js v20.x LTS
Web Framework	Express.js v4.x
Database	MongoDB v7.x (local)
ODM	Mongoose v8.x
Frontend	Vanilla HTML/CSS/JS (Chrome 120+)
Auth Library	jsonwebtoken v9.x, bcryptjs v2.x
Chart Library	Chart.js v4.x (CDN)
Operating System	Windows 11 / Ubuntu 22.04
API Testing	Thunder Client / Postman

5.2 Authentication System Results

The authentication system was tested across the following scenarios:

Test Case	Input	Expected Output	Result
Valid Registration	New email + valid password	201 Created, JWT returned	PASS
Duplicate Email	Existing email	400 User already exists	PASS
Short Password	Password < 6 chars	400 validation error	PASS
Valid Login	Correct credentials	200 + JWT + user object	PASS
Wrong Password	Incorrect password	401 Invalid credentials	PASS
No Token Access	No Authorization header	401 Not authorized	PASS
Admin Access as User	User JWT on /api/admin	403 Admin access only	PASS
Auto Admin Seed	Server start, no admin	Admin created in DB	PASS

5.3 Skill Match Engine Results

The Skill Match Engine was tested with representative skill sets against the 6 seeded job roles:

User Skills	Target Role	Match %	Match Level	Courses Recommended
Python, TensorFlow, NumPy	AI / ML Engineer	50%	Needs Improvement	PyTorch Deep Learning, ML by Andrew Ng, Pandas Tutorial
Python, SQL, Pandas, NumPy, Statistics	Data Scientist	83%	Strong Match	Power BI (not in catalogue — admin can add)
AWS, Docker, Linux	Cloud Engineer	60%	Moderate Match	Kubernetes for Beginners, Terraform Full Course
HTML, CSS, JavaScript, React, Git	Full Stack Developer	63%	Moderate Match	Node.js Full Course, MongoDB Full Course
CI/CD, Docker, Jenkins, Kubernetes, AWS, Linux	DevOps Engineer	100%	Strong Match	None (all skills matched)
Python	Cybersecurity Analyst	20%	Low Match	No matching courses (admin to add security courses)

Results confirm that the algorithm correctly identifies matched and missing skills, computes percentages accurately, and recommends relevant courses. The Cybersecurity Analyst case highlights a current limitation: the seeded course catalogue does not include Network Security or SIEM courses — this can be resolved by admins adding domain-specific courses through the Admin Dashboard.

5.4 Income Tracker Results

The income tracker was tested with a simulated 6-month income progression for an internship to full-time transition scenario:

Month	Source	Amount (INR)	Growth from First
2025-07	Internship	15,000	Baseline
2025-08	Internship	15,000	0%
2025-09	Freelance + Internship	28,000	+87%
2025-10	Freelance	32,000	+113%
2025-11	Full-Time Salary	45,000	+200%
2025-12	Full-Time Salary	45,000	+200%

The system correctly computed: Average Monthly Income = INR 30,000; Highest Income = INR 45,000; Entry Count = 6; Growth from First Entry = 200%. The Chart.js line chart rendered the progression correctly with smooth tension interpolation, demonstrating clear visual career income growth.

5.5 API Performance

API response times were measured for key endpoints under local test conditions:

Endpoint	Method	Avg Response Time	Status
/api/auth/login	POST	180ms (with bcrypt)	200 OK
/api/auth/register	POST	200ms (bcrypt hash)	201 Created
/api/jobs	GET	45ms	200 OK
/api/courses	GET	38ms	200 OK
/api/income	GET	52ms (JWT + DB query)	200 OK
/api/admin/stats	GET	95ms (3 parallel counts)	200 OK
/api/admin/jobs (POST)	POST	48ms	201 Created
/api/admin/jobs/:id (DELETE)	DELETE	42ms	200 OK

The higher response times for auth routes are attributable to bcrypt's intentional computational cost (salt rounds = 10), which is expected and desirable from a security standpoint. All non-auth endpoints respond under 100ms, ensuring responsive user experience.

5.6 UI/UX Testing

The frontend was tested across devices and themes:

- Desktop (1920x1080): All seven pages render correctly with full navigation, tables, and charts
- Tablet (768px): Responsive CSS grid collapses to 2-column layouts; hamburger menu activates
- Mobile (375px): Single-column layout; income grid, stats strip, and job cards stack vertically
- Dark Theme: All CSS variables switch correctly; text contrast ratios meet WCAG AA standards
- Light Theme: Default state; gradients and card backgrounds render cleanly
- Cross-browser: Tested on Chrome 120, Firefox 121, and Edge 120 — no rendering issues

5.7 Discussion

The experimental results demonstrate that CareerStack achieves its core objectives: the authentication system is secure and role-aware, the Skill Match Engine produces accurate, interpretable results, the income tracker correctly computes and visualises financial progression, and the admin system provides dynamic content management without requiring code changes.

The primary limitation identified during testing is the dependence of course recommendation quality on the breadth of the course catalogue. Domains with thin course coverage (e.g., Cybersecurity, Blockchain) produce fewer recommendations — an issue addressable through ongoing admin curation or future automated ingestion from external course APIs (e.g., Coursera API, edX API).

The income tracker's current use of `localStorage` for frontend-side persistence is a pragmatic choice for the current phase, but the fully functional backend Income API (`income.routes.js`, `Income.js` model) provides a ready migration path to server-side persistence with full multi-device sync capability.

CHAPTER 6: CONCLUSIONS AND FUTURE DIRECTIONS

6.1 Conclusions

CareerStack successfully demonstrates that a full-stack web application built with Node.js, Express.js, MongoDB, and vanilla JavaScript can deliver meaningful career development functionality — skill gap analysis, personalised course recommendations, income tracking, and role-based administration — within a single, lightweight, open-access platform.

The project achieves all eight stated objectives: a responsive multi-page frontend is implemented across seven HTML pages; a RESTful API backend with five route modules is operational; JWT-based authentication with bcrypt password hashing is secure and functional; the Skill Match Engine correctly computes match percentages and recommends ranked courses; the Income Tracker visualises financial progression with Chart.js; and the Admin Dashboard enables dynamic platform management.

From an SDG perspective, CareerStack directly contributes to Goal 8 (Decent Work and Economic Growth) by reducing the information asymmetry between students' skills and employer requirements, providing free access to targeted learning resources, and fostering financial literacy through income visualisation.

The zero-framework MPA architecture demonstrates that a sophisticated, feature-rich career platform can be built without heavy JavaScript frameworks, keeping the system accessible on low-bandwidth connections and deployable on minimal server infrastructure — important considerations for inclusivity in developing economies.

6.2 Limitations

- Income data is currently stored in localStorage — server-side persistence via the Income API is implemented but not yet wired to the frontend
- Social login (Google/GitHub OAuth) is planned but not implemented — buttons are present in login.html with informational alerts
- The course catalogue depends on admin curation — no automated external course API ingestion is implemented
- No email verification or password reset flow is implemented
- The skill match algorithm is keyword-based — it does not handle semantic similarity (e.g., 'ML' vs 'Machine Learning')
- No unit or integration test suite is included in the current codebase

6.3 Future Directions

Several enhancements are planned for future development phases:

20. Server-Side Income Persistence: Migrate income tracker frontend to consume the existing `/api/income` REST API, enabling multi-device sync and admin-visible income analytics
21. OAuth Integration: Implement Google and GitHub OAuth 2.0 login using Passport.js or Auth0
22. Semantic Skill Matching: Integrate a pre-trained Word2Vec or sentence-transformers model to enable fuzzy/semantic skill matching (e.g., matching 'Deep Learning' against 'Neural Networks')
23. Resume Parser: Add a PDF resume upload feature that auto-extracts skills using natural language processing, pre-populating the skill input field
24. External Course API Integration: Integrate Coursera's partner API or the YouTube Data API to dynamically fetch and index courses, eliminating manual admin curation
25. Email Notifications: Implement nodemailer-based email verification and password reset flows
26. Unit and Integration Testing: Add Jest-based test suites for API routes and Supertest for endpoint integration testing
27. Deployment: Deploy backend on Railway or Render, frontend on Netlify or Vercel, with MongoDB Atlas for the production database
28. Mobile App: Build a React Native mobile application consuming the existing REST API
29. Analytics Dashboard: Add admin-facing analytics with Chart.js — user growth trends, most-searched job roles, and popular course recommendations

6.4 Final Remarks

CareerStack represents a meaningful step toward democratising career guidance and skill development tools for students in India and beyond. By combining practical full-stack engineering with a clear social mission aligned to the UN Sustainable Development Goals, the project demonstrates that impactful, scalable technology can be built with accessible tools and a focused, user-centric design philosophy.

The codebase is structured for extensibility — each route module, model, and frontend page is independently maintainable, and the clean separation of concerns between authentication, business logic, and data access creates a solid foundation for the enhancements described above.

REFERENCES

- [1] Acemoglu, D., & Autor, D. (2011). "Skills, Tasks and Technologies: Implications for Employment and Earnings." *Handbook of Labor Economics*, 4, 1043-1171.
- [2] Deming, D. J. (2017). "The Growing Importance of Social Skills in the Labor Market." *The Quarterly Journal of Economics*, 132(4), 1593-1640.
- [3] Mongoose Documentation (2024). Mongoose v8 Official Documentation. Retrieved from <https://mongoosejs.com/docs/>
- [4] Express.js Foundation (2024). Express.js v4 API Reference. Retrieved from <https://expressjs.com/en/4x/api.html>
- [5] JSON Web Tokens (2024). Introduction to JSON Web Tokens. Retrieved from <https://jwt.io/introduction>
- [6] Chart.js (2024). Chart.js v4 Documentation. Retrieved from <https://www.chartjs.org/docs/latest/>
- [7] United Nations (2015). "Transforming our World: The 2030 Agenda for Sustainable Development." United Nations General Assembly Resolution A/RES/70/1.
- [8] NASSCOM (2024). "India's Tech Talent Skills Report 2024." NASSCOM Foundation, New Delhi.
- [9] MDN Web Docs (2024). "Using the Fetch API." Mozilla Developer Network. Retrieved from https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API/Using_Fetch
- [10] bcryptjs (2024). npm package documentation. Retrieved from <https://www.npmjs.com/package/bcryptjs>
- [11] OWASP Foundation (2024). "OWASP Top Ten Web Application Security Risks." Retrieved from <https://owasp.org/www-project-top-ten/>
- [12] Ricci, F., Rokach, L., & Shapira, B. (2015). "Recommender Systems Handbook." Springer, New York.